

FAST
FORWARD

АКОС

Лекция 8
Процессы

Процесс

Процесс - запись в таблице процессов памяти ядра

Идентификатор - PID - process ID

Содержит:

- Виртуальную память
- Таблицу файловых дескрипторов
- Параметры для планировщика

Планировщик

Цель - выбрать задачу на исполнение

Метрики:

- Честность (Fairness)
- Задержка (Latency)
- Производительность (Throughput)
- Бесконечное ожидание (Starvation)

Примеры задач:

- CPU bound вычисления
- Консольные утилиты
- Вывод звука
- Игры

Переключение контекста

Добровольное вытеснение

- IO
- Sleep
- Mutex wait
- Yield

Принудительное вытеснение

- По таймеру
- Аппаратные прерывания

Переключение контекста

Явные траты

- Сохранить регистры текущего потока
- Записать метрики в ядре
- Ввыбрать следующий
- Восстановить его регистры

Косвенные траты

- Теряется “разогретость”
- Портятся cache locality и branch predictor state
- Портится TLB cache

Планировщик Linux

- Достаточно “честный”

- Приоритезирует IO задачи

- Стандартная квота времени - несколько миллисекунд

- Старается не перемещать задачи без необходимости

- Отделяет “обычные” задачи от задач “реального времени”

ps

Информация о процессах

a указывает ps вывести на дисплей процессы всех пользователей, за исключением тех процессов, которые не связаны с терминалом и процессами группы лидеров.

x в ps перечисляет процессы без управляющего терминала. В основном это процессы, которые запускаются во время загрузки и работают в фоновом режиме.

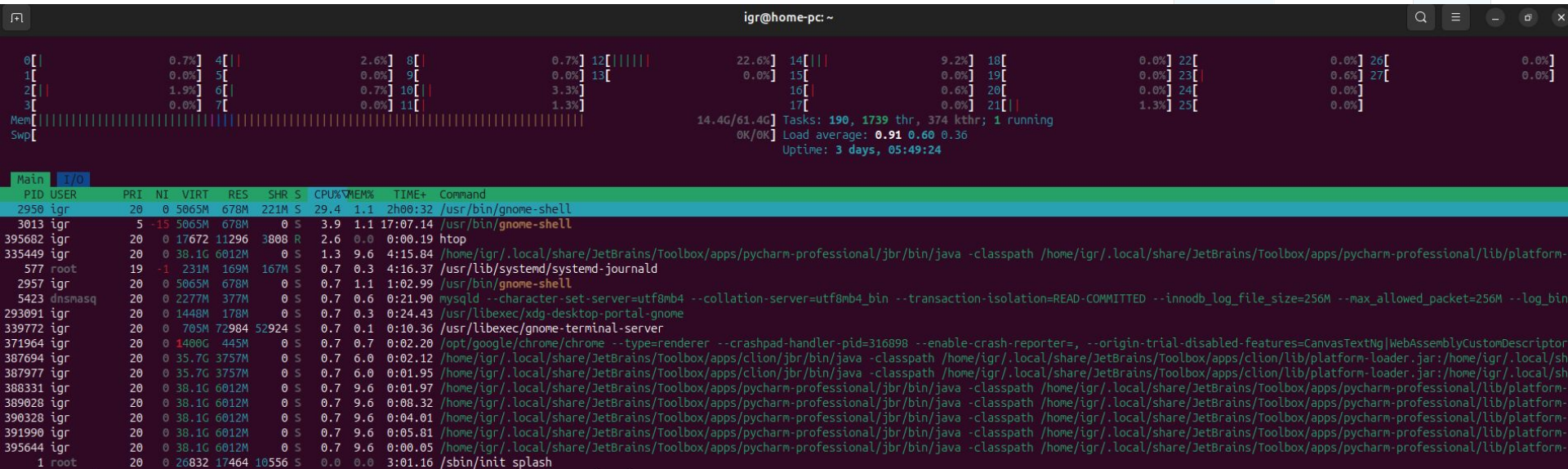
e отобразить все процессы.

f подробная информация о процессах.



FAST
FORWARD

htop/top



Статусы процесса

- R - running/runnable, выполняется/готов к исполнению
- S - sleeping, ожидает возвращения из системного вызова
- T – stopped, остановлен пользователем
- Z – zombie
- t – остановлен отладчиком
- D - uninterruptible sleep



FAST
FORWARD

Дерево процессов

```
igr@home-pc:~$ pstree
systemd─ModemManager─3*[{ModemManager}]
        └─NetworkManager─3*[{NetworkManager}]
        └─accounts-daemon─3*[{accounts-daemon}]
        └─avahi-daemon─avahi-daemon
        └─blkmapped
        └─bluetoothd
        └─colord─3*[{colord}]
        └─containerd─20*[{containerd}]
        └─containerd-shim─mysqld─36*[{mysqld}]
                        └─11*[{containerd-shim}]
        └─cron
        └─cups-browsed─3*[{cups-browsed}]
        └─cupsd
        └─dbus-daemon
        └─dockerd─193*[{dockerd}]
        └─fwupd─3*[{fwupd}]
        └─gdm3─gdm-session-wor─gdm-wayland-ses─gnome-session-i─3*[{gnome-session-i}]
                └─4*[{gdm3}]          └─3*[{gdm-session-wor}]          └─3*[{gdm-wayland-ses}]
        └─ngatetun
        └─nvidia-persiste
        └─ollama─30*[{ollama}]
        └─polkitd─3*[{polkitd}]
        └─power-profiles─3*[{power-profiles-}]
        └─rpcbind
        └─rsyslogd─3*[{rsyslogd}]
        └─rtkit-daemon─2*[{rtkit-daemon}]
        └─smartd
        └─snapd─34*[{snapd}]
        └─switcheroo-cont─3*[{switcheroo-cont}]
        └─systemd─(sd-pam)
```

Сироты

zombie = ребёнок умер, родитель жив и не забрал статус

orphan = родитель умер, ребёнок жив

Reparenting

Перепривязка к PID = 1

Subreapers

Процесс детдом

procfs

- Псевдо-файловая система

- Содержимое генерируется ядром на лету

- Через неё можно смотреть процессы, память, дескрипторы, параметры ядра

- полезно:

- status, stat, cmdline, fd, maps, task, cwd, etc...

fork

```
pid_t fork(void);
```

Полная копия памяти - copy on write

Копия дескрипторов *

0 - новому, pid - старому

-1 - ошибка

Пример

```
pid_t pid = fork();  
if (pid == 0) {  
    printf("child %d\n", getpid());  
} else {  
    printf("parent %d\n", getpid());  
}
```

Пример 2

```
int main() {  
    printf("hello\n");  
    fork();  
}
```

stdout - буферизированный

Копия памяти - копия буфера

Полезные системные вызовы

Подождать секунды

```
unsigned int sleep(unsigned int seconds);
```

До получения сигнала

```
int pause(void);
```

Дождаться дочернего процесса

```
pid_t wait(int * _Nullable wstatus);
```


Пример 3

```
int main () {  
    pid_t id = fork();  
    if (id == 0) {  
        printf("child\n");  
        sleep(2);  
    } else {  
        printf("parent\n");  
    }  
    printf("end\n");  
}
```

Пример 4

```
int main () {  
    pid_t id = fork();  
    if (id == 0) {  
        printf("child\n");  
        sleep(2);  
    } else {  
        printf("parent\n");  
    }  
    printf("end\n"); wait(NULL);  
}
```

Таблица файловых дескрипторов

fork - копия таблицы

struct file - ссылается на ту же структуру в ядре - seek общий

close - закрыть только в своей таблице

lseek - меняет в обоих

Пример 5

```
int main () {  
    FILE * file = fopen("out.txt", "w");  
    int pid = fork();  
    if (pid == 0) {  
        fprintf(file, "child\n");  
    } else {  
        fprintf(file, "parent\n");  
    }  
}
```

Не атомарность проверки

- Процесс 1

```
fd=open("A", O_RDONLY, 0);  
if(fd<0 && errno=ENOENT) {  
    fd=open("A", O_WRONLY  
    |O_CREAT, 0600);  
}
```

- Между операцией проверки на существование и создания файла может вклиниться другой процесс и создать свой файл

- Процесс 2

```
fd=open("A", O_CREAT|  
O_WRONLY|O_TRUNC, 0666);
```

- В старых Unix это был один из способов для получения прав root в случае ошибки в привилегированном ПО

Решение

- x Open the file exclusively (like the O_EXCL flag of open(2)). If the file already exists, fopen() fails, and sets errno to EEXIST. This flag is ignored for fdopen().

```
fopen("out.txt", "wx")
```

И еще

□ POSIX (стандарт) - не даёт гарантий атомарности на запись или чтение из файла

□ Например, результатом `write("123")` и `write("abc")` из разных процессов может быть "1a2b3c"

□ Linux - даёт гарантию "атомарности"

Замещение процесса

Семейство вызовов `exec*`

- PID не меняется
- Создаётся новое пространство виртуальной памяти
- В данную виртуальную память загружается бинарный файл и начинается его исполнение



exec

```
int execl(const char *path, const char *arg, ...);  
int execlp(const char *file, const char *arg, ...);  
int execl_e(const char *path, const char *arg, ..., char * const envp[]);  
int execv(const char *path, char *const argv[]);  
int execvp(const char *file, char *const argv[]);  
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

Пример 1

```
int main() {  
    execlp("ls", "ls", "-la", NULL);  
    perror("Error\n");  
}
```

Пример 2

```
int main() {  
    execl("/usr/bin/ls", "ls", "-la", NULL);  
    perror("Error\n");  
}
```

Пример 3

```
int main() {  
    pid_t child = fork();  
    if (child == 0) {  
        execv(...); // Child  
    }  
    int status;  
    wait(&status);  
}
```



FAST
FORWARD

wait

```
pid_t wait(int *_Nullable wstatus);
```

```
pid_t waitpid(pid_t pid, int *_Nullable wstatus, int options);
```

WIFEXITED(wstatus)

WEXITSTATUS(wstatus)

WIFSIGNALED(wstatus)

WTERMSIG(wstatus)

dup

```
int dup(int oldfd);
```

```
int dup2(int oldfd, int newfd)
```

Дублирует один дескриптор в другой.

Если newfd существует - закрывает

Пример

```
pid_t child = fork();  
if (child == 0) {  
    int fd = open(...);  
    dup2(fd, STDOUT_FILENO);  
    exec(...);  
    _exit(127);  
}
```



FAST
FORWARD

AKOC

QCourse.ru 2026